MODULE 15 · DOMAIN 2 - ML MODEL AND FM DEVELOPMENT

# Tuning and regularisation

Four search strategies, and the remedies for a model that has learned too much.

MODULE 15 of 43 DOMAIN 2 - Model Dev WEIGHT 24%

admin@sthithapragna.com

Parameters are **learned**. Hyperparameters are **chosen before training begins**.

# Hyperparameters, grouped by what they control

| GROUP | EXAMPLES |
|---|---|
| Model capacity | max_depth, number of layers, number of trees, k in K-Means |
| Optimisation | learning rate, batch size, momentum, optimiser choice |
| Regularisation | alpha, lambda, dropout rate, weight decay, gamma |
| Training length | epochs, num_round, early stopping patience |
| Data handling | subsample, colsample_bytree, class weights |

ON THE EXAM

Capacity and regularisation pull against each other. Most tuning is finding the balance between them.

# What a tuning job actually does

01
### Define
Ranges for each hyperparameter, and an objective metric.

›

02
### Search
A strategy picks configurations to try.

›

03
### Train
Each configuration runs as a full training job.

›

04
### Compare
Objective metric decides which won.

›

05
### Deploy
The best training job's model is the one you take.

ON THE EXAM

Every trial is a complete training job. That is why a 50-job tuning run costs 50 training jobs.

# Choosing the objective metric

- Tuning optimises **one number**. Everything else about the model is incidental to it.
- Optimising **accuracy** on an imbalanced problem selects a model that predicts the majority class.
- Use a **validation** metric, never a training one, or you select the most overfitted trial.
- Set **objective_type** correctly: Maximize for AUC or F1, Minimize for RMSE or loss.
- Module 17 covers which metric a problem actually needs. This is where that choice gets spent.

> ON THE EXAM
>
> A tuning job is a wish granted literally. It will find the configuration that maximises exactly what you asked for.

# Reproducibility of a tuning job

- A **random seed** can be set on the tuning job, and the effect differs by strategy.
- **Random search and Hyperband:** the same seed gives up to 100 percent reproducibility.
- **Bayesian:** the same seed only **improves** reproducibility. It does not guarantee it.
- **Grid search** needs no seed: the same ranges give identical optimal values by construction.
- Module 14's caveat applies too. Distributed training adds non-determinism of its own.

ON THE EXAM

If a scenario demands that a tuning result be exactly reproducible for audit, grid search is the strategy that guarantees it.

# Parameter types and ranges

- **ContinuousParameter** for any value in a range, such as a learning rate.
- **IntegerParameter** for whole numbers, such as max_depth.
- **CategoricalParameter** for a fixed set, such as optimiser choice.
- **Up to 30 hyperparameters** can be tuned at once, and fewer is usually better.
- Ranges that are too wide waste jobs. Ranges that are too narrow may exclude the answer.

> ON THE EXAM
>
> Every extra hyperparameter multiplies the space to search. Tune the few that matter, not everything available.

# Logarithmic scaling

- Some hyperparameters matter **multiplicatively**, not additively.
- The gap from 0.0001 to 0.001 matters far more than the gap from 0.1 to 0.101.
- **Logarithmic scaling** samples evenly across orders of magnitude instead of evenly across the range.
- Learning rate and regularisation strength are the classic cases.
- Scaling types include Auto, Linear, Logarithmic and ReverseLogarithmic.

> ON THE EXAM
>
> With linear sampling over 0.0001 to 1, almost every sampled value is large. The interesting region never gets explored.

# How each one searches

| STRATEGY | HOW IT BEHAVES |
|---|---|
| Bayesian | Learns from prior runs to choose the next. Sequential, so it cannot massively scale |
| Random | Draws independently. Runs the largest number of parallel jobs of any strategy |
| Hyperband | Uses intermediate results to stop weak runs and reallocate epochs to strong ones |
| Grid | Tries every combination. Reproducible by construction, and expensive |

ON THE EXAM

Bayesian is the default. Hyperband is the answer for iterative algorithms. Random is the answer when you want massive parallelism.

# Bayesian optimisation

- Builds a model of the objective as a function of the hyperparameters, and uses it to pick the next trial.
- **It gets better as it goes**, because each result informs the next choice.
- The most efficient strategy when the job budget is small. Fewer trials for the same result.
- **Sequential by nature**, so high parallelism wastes it. Parallel jobs cannot learn from each other.
- A random seed improves reproducibility here, but does not guarantee it.

> ON THE EXAM
>
> The trade-off is direct: more parallel jobs means faster wall-clock and a less informed search.

# Random search

- Draws hyperparameter values randomly from the ranges. No learning between trials.
- Because trials are independent, it **runs the largest number of parallel jobs** of any strategy.
- Right when wall-clock time matters more than job efficiency, and capacity is available.
- It wastes trials on regions a Bayesian search would have abandoned.
- A fixed random seed gives up to 100 percent reproducibility for the same tuning job.

> ON THE EXAM
>
> Random search is not naive. On a large space it reliably beats grid search, because most hyperparameters do not matter much.

# Hyperband

- **Multi-fidelity:** it uses intermediate results, not only final ones.
- It **stops underperforming runs early** and reallocates epochs to promising configurations.
- It scales to many parallel jobs, so it combines efficiency with parallelism.
- **The condition:** it only works on iterative algorithms that publish results at different resource levels.
- That means an algorithm emitting a metric **per epoch**. A single-shot algorithm has nothing for it to read.

> ON THE EXAM
>
> AWS recommends Hyperband for large jobs. The per-epoch metric requirement is the thing that rules it in or out.

# Grid search

- Methodically tries **every combination** of the values you specified.
- **Reproducible by construction:** the same hyperparameters give identical optimal values every time.
- Right when simplicity and transparency of the search matter, or you must reproduce a result.
- Cost explodes combinatorially. Three parameters with five values each is 125 training jobs.
- It is the only strategy where max_jobs is not specified by default, since the grid defines it.

> ON THE EXAM
>
> Choose grid for auditability, not for efficiency. Its selling point is that you can explain exactly what was tried.

# Bayesian or Hyperband?

BAYESIAN, WHEN

## The budget is small

- Few trials available, each one must count.
- The algorithm reports only a final metric.
- Training runs are short.
- You accept lower parallelism.

HYPERBAND, WHEN

## The jobs are large

- Each training run is long and expensive.
- The algorithm emits a metric per epoch.
- You want high parallelism as well.
- Stopping weak runs early saves real money.

THE TIEBREAKER

Does the algorithm publish intermediate results? If not, Hyperband cannot work, however large the jobs are.

# max_jobs and max_parallel_jobs

- **max_jobs** is the total budget. **max_parallel_jobs** is how many run at once.
- Both default to **1** for every strategy except Grid. Leaving them alone runs one job.
- **High parallelism finishes sooner. Low parallelism searches better**, on Bayesian.
- Setting max_parallel equal to max_jobs on Bayesian reduces it to random search.
- Both count against your training instance quotas from module 14.

> ON THE EXAM
>
> This is the single most common AMT misconfiguration: full parallelism on a Bayesian job, which discards its only advantage.

# What a finished tuning job tells you

- The **best training job** is identified, with its hyperparameters and objective value.
- Every trial is listed, so you can see the **shape** of the search, not only the winner.
- **Look at where good trials cluster.** That tells you where to narrow the ranges next.
- A winner at a range boundary means the optimum is probably outside the range.
- Results are queryable as a dataframe, so you can plot objective against each hyperparameter.

> ON THE EXAM
>
> The losing trials carry most of the information. They tell you which hyperparameters mattered and which did nothing.

# Two-stage tuning

- **Stage one:** wide ranges, cheap settings, modest job count. Find the promising region.
- **Stage two:** warm start from stage one, narrow ranges around the winners, more jobs.
- This finds better models for less money than one large search over wide ranges.
- Use a smaller data sample in stage one if training is slow. Full data in stage two.
- Tuning once on wide ranges and accepting the result is the common and wasteful pattern.

> ON THE EXAM
>
> Tuning is iterative, exactly like fine-tuning in module 11. One pass is a first draft, not an answer.

# Early stopping

- **early_stopping_type** is either **Off** or **Auto**.
- With Auto, AMT predicts from earlier evaluations whether a trial is promising, and stops it if not.
- It saves compute directly, which on long training jobs is the largest saving available.
- Hyperband has its own stopping mechanism built into the strategy itself.
- Distinct from early stopping **inside** a training run, which halts on rising validation loss.

> ON THE EXAM
>
> Two different things share the name. AMT early stopping kills a whole trial; in-training early stopping ends one run at its best epoch.

# The tuning step in SageMaker Pipelines

- A **tuning step** runs a hyperparameter tuning job as part of a pipeline, with no custom integration code.
- So retraining can **re-tune** rather than reusing hyperparameters chosen months ago on different data.
- The best model from the step feeds the next step, typically evaluation and registration.
- Re-tuning on every run is expensive. Many teams tune periodically and retrain more often.
- Module 28 covers pipelines properly. This is the hook that connects them.

> ON THE EXAM
>
> Hyperparameters chosen on last year's data are a quiet form of drift. Nothing alerts you that they stopped being optimal.

# Warm start

- Seeds a new tuning job with the evaluations from one or more **parent jobs**.
- **IDENTICAL_DATA_AND_ALGORITHM:** same data and algorithm, so results transfer directly.
- **TRANSFER_LEARNING:** the data or algorithm changed, so results are treated as a weaker prior.
- The usual pattern: run a broad search, then warm start a **narrower** search around what won.
- Without warm start, every tuning job rediscovers what the last one already learned.

> ON THE EXAM
>
> Two-stage tuning is the practical pattern: wide and cheap first, then narrow and focused, seeded by the first.

# Autotune

- You supply an objective metric. It chooses the rest.
- It selects **ranges, strategy, maximum runtime, early stopping type, retry count** and the convergence flag.
- Available in the CreateHyperParameterTuningJob API and in the Python SDK tuner.
- You can **review and override** anything it chose, so it is a starting point rather than a black box.
- Right when nobody on the team knows what sensible ranges look like for this algorithm.

> ON THE EXAM
>
> Autotune is to tuning what Autopilot is to algorithm selection: a defensible default that you can inspect and adjust.

# Which strategy

| WHAT THE SCENARIO SAYS | THE STRATEGY |
|---|---|
| Nothing specific. A sensible default | Bayesian |
| Deep learning, metrics per epoch, long runs | Hyperband |
| Maximum parallelism, capacity available | Random |
| Must reproduce a previous tuning result | Grid |
| Auditability of what was tried matters | Grid |
| Small job budget, each trial must count | Bayesian |
| Nobody knows what ranges to set | Autotune |
| A previous tuning job already exists | Warm start, whichever strategy |

ON THE EXAM

Rows two and four are the ones that override the default. Everything else usually lands on Bayesian.

Now the problem tuning is usually trying to fix: a model that **memorised** rather than learned.

# Overfitting and underfitting

- **Overfitting:** training error low, validation error much higher. The gap is the symptom.
- **Underfitting:** both errors high. The model cannot represent the pattern at all.
- **The gap between the two curves is what you read**, not either curve alone.
- Overfitting worsens with more capacity, more epochs, and less data.
- Underfitting improves with more capacity, better features, and longer training.

> ON THE EXAM
>
> The remedies are opposites. Applying the overfitting fix to an underfitting model makes it worse, and vice versa.

# The remedies, and what each costs

| REMEDY | HOW IT WORKS |
|---|---|
| More training data | The best fix, and usually the least available |
| Reduce capacity | Shallower trees, fewer layers, fewer parameters |
| L1 regularisation (alpha) | Penalises weight magnitude, drives some weights to exactly zero |
| L2 regularisation (lambda) | Penalises squared weights, shrinks all of them smoothly |
| Dropout | Randomly disables neurons during training. Neural networks only |
| Early stopping | Stop at the epoch where validation loss bottomed out |
| Data augmentation | Synthetic variation, from module 05. Effectively more data |
| Subsampling | Each tree sees a fraction of rows or columns. Tree ensembles |

ON THE EXAM

L1 also performs feature selection, because weights driven to exactly zero remove that feature from the model entirely.

# Tuning on an imbalanced problem

- Put **scale_pos_weight** in the search space for XGBoost, and let tuning find the balance.
- Do not optimise accuracy. Optimise **AUC, F1, or recall**, depending on what module 17 says the problem needs.
- Resampling from module 08 applies to the **training split only**, inside each trial.
- Keep validation at the real class distribution, or every trial is scored against a fiction.
- Threshold adjustment happens after tuning, not during it. Tuning finds the model; you choose the cut-off.

> ON THE EXAM
>
> Imbalance affects three things at once here: the objective metric, the search space, and how you split. Miss one and the others do not save you.

# L1 or L2?

L1, OR LASSO

## Drives weights to zero

- Penalises the absolute value of weights.
- Some weights become exactly zero.
- That removes features entirely.
- Good when many features are irrelevant.

L2, OR RIDGE

## Shrinks weights smoothly

- Penalises the square of weights.
- Weights get small, rarely exactly zero.
- All features stay in the model.
- Good when most features carry some signal.

THE TIEBREAKER

If a question wants a sparse model, or automatic feature selection, that is L1. If it wants stability, that is L2.

# Dropout, and friends

- **Dropout** randomly disables a fraction of neurons on each training step.
- That stops the network relying on any one path, so it must learn redundant representations.
- It is applied during **training only**. At inference the full network is used.
- Typical rates are 0.2 to 0.5. Too high and the network cannot learn at all.
- **Batch normalisation** and **weight decay** also regularise, as side effects of their main job.

> ON THE EXAM
>
> Dropout is the neural network answer specifically. Offering it for an XGBoost model is a category error.

# Fixing underfitting

- **Increase capacity:** deeper trees, more layers, more estimators.
- **Train longer:** more epochs or more boosting rounds.
- **Reduce regularisation.** You may have constrained the model too hard.
- **Better features** usually beat all of the above, from module 05.
- More data does not fix underfitting. The model already cannot use what it has.

> ON THE EXAM
>
> The last row is the useful one: more data is the answer to overfitting, and does nothing at all for underfitting.

# What a tuning job costs

| FACTOR | EFFECT ON COST |
|---|---|
| max_jobs | Direct multiplier. 50 jobs costs 50 training runs |
| Early stopping Auto | Cuts cost by killing unpromising trials part-way |
| Hyperband | Cuts cost further, by stopping weak runs after a few epochs |
| Warm start | Fewer jobs needed, because the search starts informed |
| Number of hyperparameters | More dimensions means more jobs to cover the space |
| Spot instances | Apply to tuning jobs too, with checkpointing. Module 14 |

ON THE EXAM

Tuning is the most expensive thing in Domain 2, because it multiplies the cost of everything in module 14.

# Symptoms, and what they usually mean

| SYMPTOM | USUAL CAUSE |
|---|---|
| Tuning job reports no objective metric | Metric definition regex never matched. Module 14 |
| Bayesian search no better than random | max_parallel_jobs set equal to max_jobs |
| Hyperband performs no better than Bayesian | Algorithm does not publish per-epoch metrics |
| Best trial is at the edge of a range | The range excluded the real optimum. Widen it |
| Tuning cost far above expectation | Early stopping off, or too many hyperparameters |
| Tuned model still overfits | Regularisation not in the search space at all |
| Only one training job ran | max_jobs left at its default of 1 |

ON THE EXAM

Row four is worth acting on. A winner sitting at a range boundary is telling you the answer is outside the range.

Tuning finds the best settings **within the space you defined**. It cannot look outside it.

A team tunes an image classification neural network. Each training run takes about six hours and publishes validation accuracy after every epoch. They have generous GPU capacity and want results by tomorrow.

They expect many hyperparameter combinations to be obviously poor within the first few epochs.

## Which tuning strategy fits best?

A Bayesian optimisation with max_parallel_jobs set to 1

B Grid search across the full hyperparameter combination set

C Hyperband, which reallocates resources using intermediate results

D Random search with early stopping set to Off

◆ PAUSE AND DECIDE

# Which tuning strategy fits best?

**A** Bayesian optimisation with max_parallel_jobs set to 1

*Fully sequential six-hour jobs will not finish by tomorrow, and it ignores the available capacity and the per-epoch metrics entirely.*

**B** Grid search across the full hyperparameter combination set

*Combinatorial cost on six-hour jobs, and it runs every poor combination to completion. It is the reproducible strategy, not the fast one.*

**C** Hyperband, which reallocates resources using intermediate results

Long jobs, per-epoch metrics, spare capacity and obviously-poor trials. All four conditions AWS names for Hyperband are present.

**D** Random search with early stopping set to Off

*Maximum parallelism, and Off means every weak six-hour run finishes in full. It discards the saving the scenario is pointing at.*

THE SIGNAL

'Publishes validation accuracy after every epoch' is the enabling condition. Without that sentence, Hyperband would be wrong.

> A team tunes an XGBoost model. Each training job takes four minutes. Their budget allows 30 training jobs in total, and the algorithm reports a single final validation AUC rather than per-round metrics.
>
> They want the most accurate model achievable within those 30 jobs.

## What should they configure?

A Bayesian optimisation, with max_parallel_jobs kept low

B Hyperband, to stop weak trials early and save jobs

C Random search with max_parallel_jobs set to 30

D Grid search over a 30-combination grid

◆ PAUSE AND DECIDE

## What should they configure?

**A** Bayesian optimisation, with max_parallel_jobs kept low

Small budget where each trial must count, and short runs so sequential search is affordable. Low parallelism preserves the learning between trials.

**B** Hyperband, to stop weak trials early and save jobs

*It needs intermediate results, and the scenario states a single final metric. There is nothing for it to act on part-way through.*

**C** Random search with max_parallel_jobs set to 30

*Finishes in four minutes and wastes the budget. Thirty independent draws with no learning is the least informed use of a small budget.*

**D** Grid search over a 30-combination grid

*Reproducible and blind. It would spend the whole budget on a fixed lattice that may miss the good region entirely.*

THE SIGNAL

Contrast with scenario 1. Same question, opposite answer, because the runs are short and there are no intermediate metrics.

A team configures a Bayesian tuning job with max_jobs set to 40 and max_parallel_jobs also set to 40, to finish as quickly as possible. All 40 jobs launch at once and complete together.

The resulting best model is no better than one they obtained previously from an unguided random search of the same size.

## What explains this?

A Bayesian optimisation requires at least 50 jobs to be effective

B All 40 jobs launched before any result existed, so none could inform another

C Bayesian optimisation is not supported for parallel execution

D The objective metric was not defined, so trials were ranked randomly

◆ PAUSE AND DECIDE

## What explains this?

A. Bayesian optimisation requires at least 50 jobs to be effective

*No such threshold exists. Bayesian is specifically the efficient choice at small budgets, which is the opposite of this claim.*

B. All 40 jobs launched before any result existed, so none could inform another

Bayesian's advantage is sequential learning. With full parallelism there is no prior information, so it degenerates into random sampling.

C. Bayesian optimisation is not supported for parallel execution

*It is supported and commonly used with moderate parallelism. The issue is degree, not support: some parallelism is a trade-off, full parallelism removes the benefit.*

D. The objective metric was not defined, so trials were ranked randomly

*A missing metric produces a tuning job that reports nothing, not one that returns a reasonable model matching random search.*

THE SIGNAL

The fix is to lower max_parallel_jobs, accepting a longer wall-clock time in exchange for a search that actually learns.

> An XGBoost fraud model reaches 0.99 AUC on training data and 0.71 on validation. The team tuned max_depth, eta and num_round, and the winning configuration used the deepest trees and the most rounds in their ranges.
>
> They propose widening the max_depth range further, reasoning that the search wanted deeper trees.

## How should you respond?

A. Agree, and widen max_depth. The winner at the boundary means the range was too narrow

B. Add dropout to the search space to regularise the model

C. Switch strategy from Bayesian to Hyperband for a more thorough search

D. Add alpha, lambda, gamma and subsample to the search space, and narrow max_depth

◆ PAUSE AND DECIDE

## How should you respond?

A Agree, and widen max_depth. The winner at the boundary means the range was too narrow

*The boundary heuristic is real, and misapplied here. A 0.28 train-validation gap is severe overfitting, and deeper trees will widen it.*

B Add dropout to the search space to regularise the model

*Right instinct, wrong algorithm. Dropout disables neurons in a neural network. XGBoost is a tree ensemble and has no neurons.*

C Switch strategy from Bayesian to Hyperband for a more thorough search

*Changes how the space is searched, not what is in it. Hyperband also needs per-round metrics, and the space still contains no regularisation.*

D Add alpha, lambda, gamma and subsample to the search space, and narrow max_depth

The search space contained only capacity parameters, so tuning could only increase capacity. Adding regularisation gives it something to trade against.

THE SIGNAL

Tuning optimised faithfully within the space it was given. That space had no way to express 'simpler model', so it never proposed one.

# What to take from this module

- **Bayesian is the default. Hyperband needs per-epoch metrics.** That condition rules it in or out.
- **Full parallelism destroys Bayesian**, because nothing can learn from anything else.
- **Grid for reproducibility. Random for maximum parallelism.**
- **L1 zeroes weights and selects features. L2 shrinks them smoothly.** Dropout is neural networks only.
- **Tuning cannot search outside the space you defined.** No regularisation in the ranges means no regularisation.

> ON THE EXAM
>
> Module 16 moves to foundation models: selecting, customising and choosing between fine-tuning, distillation and prompting.